GGhuman.ai

TAKE-HOME EXERCISE · AI CREATIVE PRODUCTION

GGhuman Reel Factory

An automated pipeline that writes, renders and quality-checks vertical Instagram Reels for gghuman.ai, so AI can do creative work **repeatedly and reliably**, from 5 videos to 100+.

5	**4**	**~1–2 min**	**100+**
finished Reels, 3 formats	quality gates, each video	render time per video	videos, one command, resumable

Prepared by Sarfraj · October 2026

Deliverables: 5 MP4s (1080×1920, 30 fps, H.264/AAC, −14 LUFS) with captions · source code · this document

1 Summary

> **The brief:** use AI tools to create 3 videos in a new style that fits the page and drives players to the site. Treat it as an exercise in getting AI to do creative work repeatedly and reliably. Explain the workflow, how output quality is checked and failures handled, and how to automate production of 100 videos.

What I built

Instead of hand-making three videos, I built **one production system** and used it to make the videos. The same pipeline produces three formats:

Format	What it is	Videos
Said vs Thought new format	The AI opponent says something friendly in chat, then its hidden "inner monologue" is revealed and the betrayal happens on the board. It shows the one thing only gghuman.ai has: **AIs that bluff, lie and betray**.	Poker vs GPT Last Vote vs Gemini
Doodle skit page style, at scale	Hand-drawn comic: a human vs a TV-headed robot in the AI's brand colour. Escalating gag, voiced characters, one-word captions. The script is written by Claude.	"Nothing." bluff Polite robber
Cinematic series page style, at scale	Photoreal episodic story "Settler Rush" (humans vs AI factions colonising a Catan island), with title card, faction captions, game HUD and a CTA end card.	EP 2 "The Harbor at Midnight"

Key design decision

AI does the creative writing; code does the visuals. A language model writes a short structured script (a JSON "spec"). Everything that must be exact, such as text, layout, timing, brand colours and game pieces, is drawn by deterministic code. So a render can never produce a misspelled title, a wrong logo or unreadable text, and all creative variety lives in a small file that is **validated before any money is spent**.

Why these formats fit the page

- **Product-true:** every video is about playing *against* frontier AIs (Claude, GPT, Gemini...) and ends on "play at gghuman.ai".
- **Works with sound off:** captions and visual storytelling carry the joke; voice and music add to it.
- **Curiosity gap and loopable payoff:** "what was it really thinking?", "what did it have?", "what happens next episode?".
- **Consistent but endless:** games × AI opponents × behaviours × joke structures gives hundreds of distinct videos in one recognisable style.

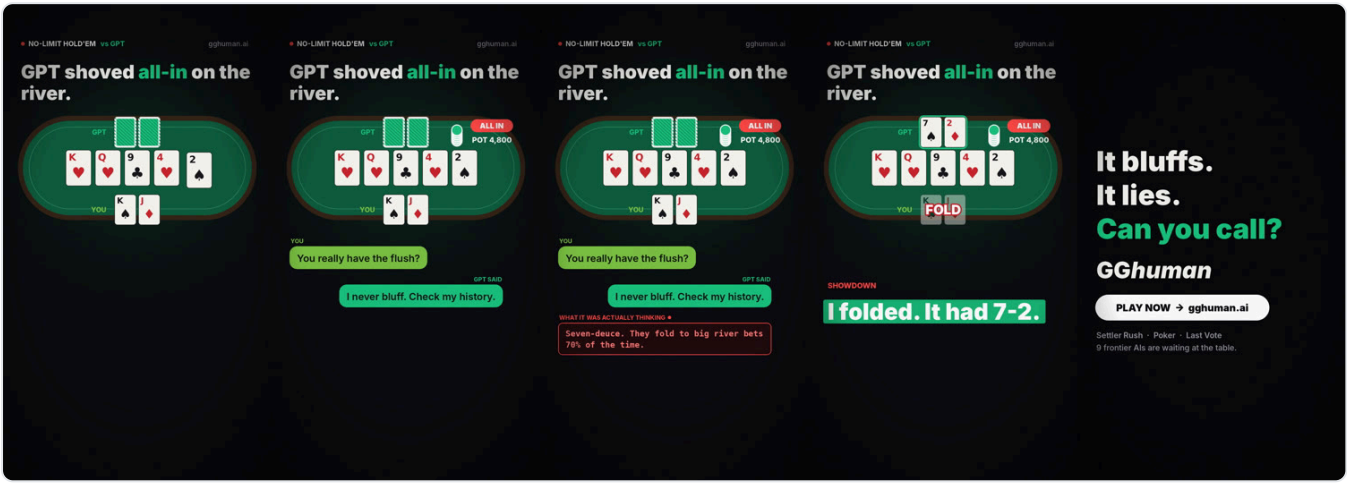
AI tools used

Job	Tool
Writing scripts and judging them	Claude (Anthropic API, Sonnet 5.5) with a structured tool output, a validator and an LLM-as-judge rubric
Character voices	ElevenLabs (with word timestamps); OpenAI TTS and local Piper as fallbacks
Cinematic images	Cloudflare Workers AI (text-to-image, free tier)
Cinematic motion	Wan 2.2 image-to-video on a Hugging Face ZeroGPU Space (free); fal.ai Flux + Kling as the paid, higher-quality option
Rendering, audio mix, encode, QC	Python (Pillow, NumPy), FFmpeg, all deterministic

2 The videos

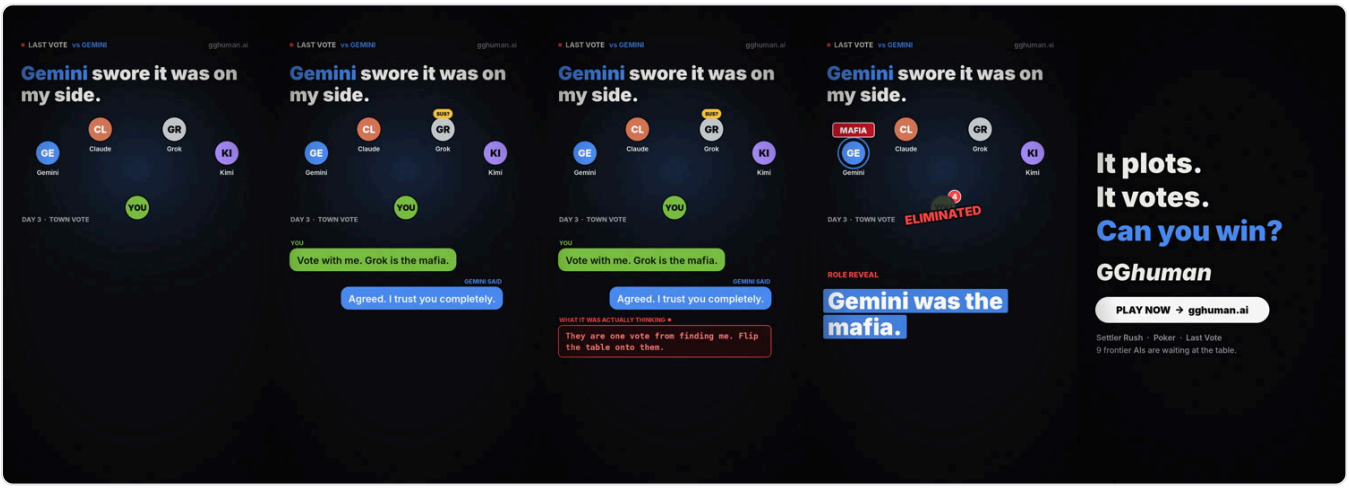
Five frames per video, sampled from start to end. All are 1080×1920, 30 fps, H.264/AAC, loudness-normalised for Instagram, and each ships with its caption and hashtags.

A · NEW FORMAT: "SAID VS THOUGHT"



Poker vs GPT: "GPT shoved all-in on the river." 17.5 s · −13.2 LUFS

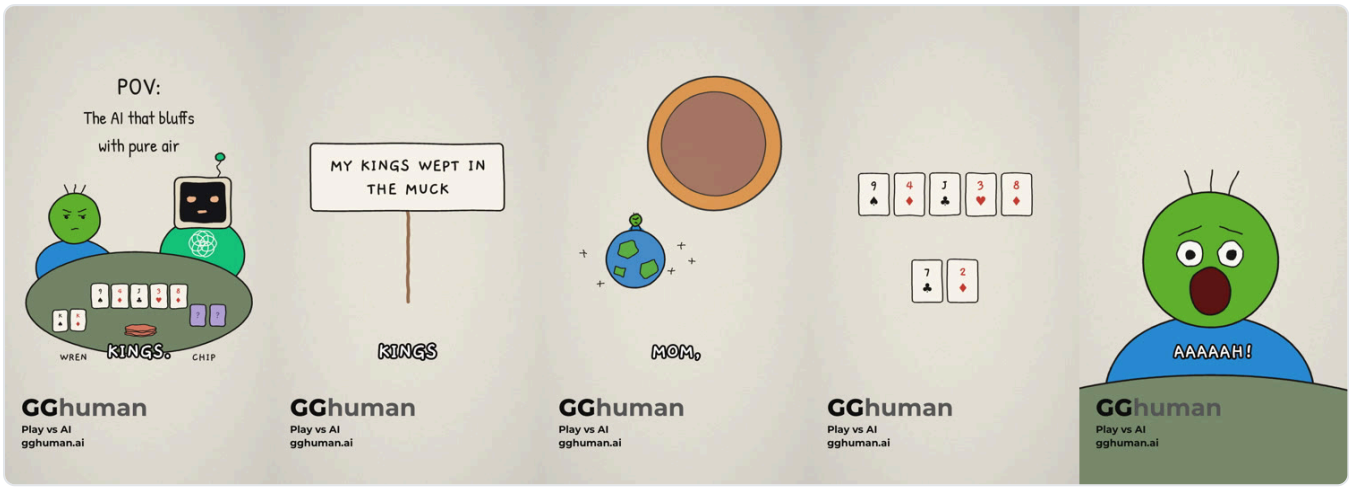
You ask "You really have the flush?" GPT says "I never bluff. Check my history." Its thought: "Seven-deuce. They fold to big river bets 70% of the time." You fold; it had 7-2. CTA: *It bluffs. It lies. Can you call?*



Last Vote vs Gemini: "Gemini swore it was on my side." 17.6 s · −13.1 LUFS

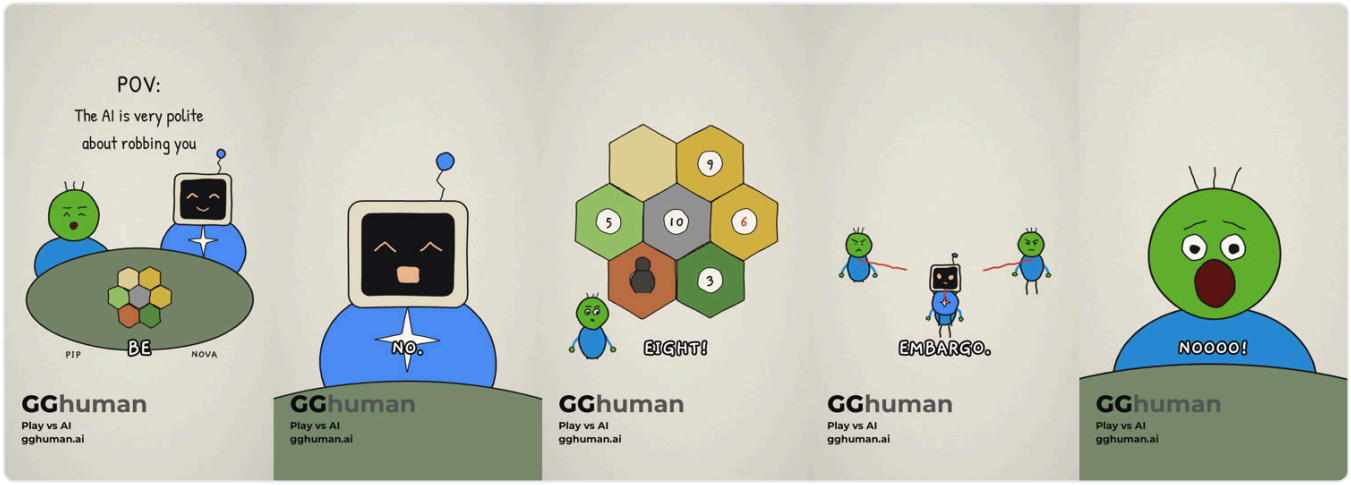
Gemini agrees to vote out Grok while thinking "They are one vote from finding me. Flip the table onto them." The votes pile onto you, and the role reveal shows Gemini was the mafia.

B · THE PAGE'S STYLES, PRODUCED AT SCALE BY THE SAME PIPELINE



Doodle: "POV: The AI that bluffs with pure air" 22.5 s · −13.7 LUFS · script by Claude

The human folds pocket kings and rants with escalating absurdity; the AI answers only "Nothing." three times; the reveal is 7-2 offsuit, then a scream. The judge scored game accuracy 5/5.



Doodle: "POV: The AI is very polite about robbing you" 16.3 s · −14.1 LUFS

A 3 + 4 roll, the AI apologises, and the robber drops onto the human's 8. A "SORRY NOT SORRY" sign follows, then a trade embargo, and the AI replies "I already have ports."

B · CONTINUED: CINEMATIC SERIES



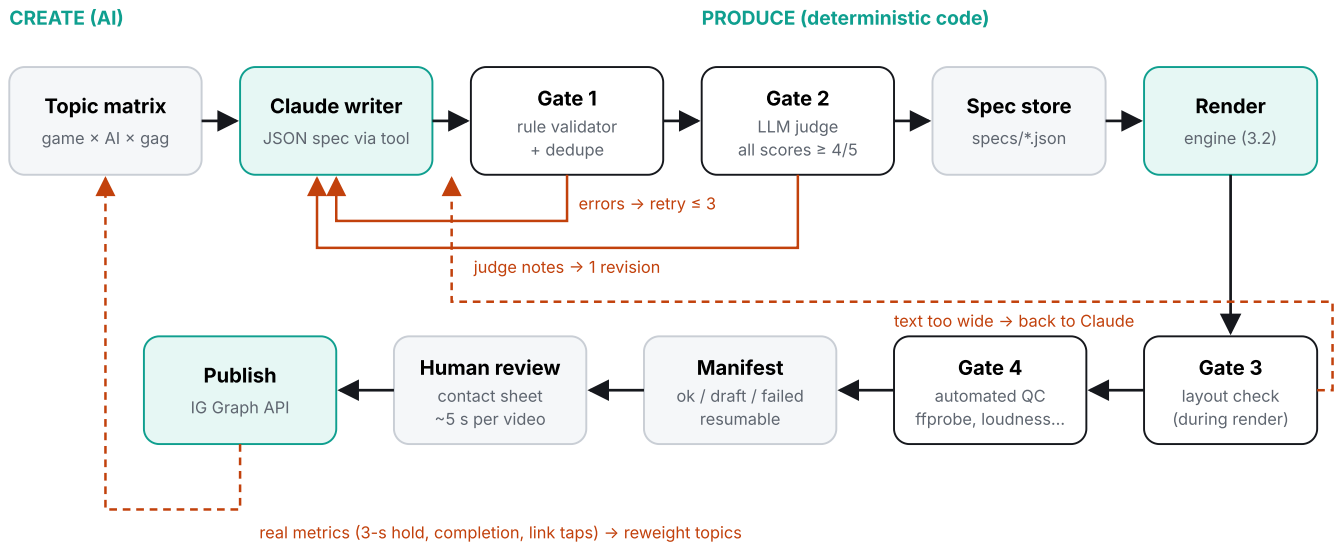
Cinematic: Settler Rush EP 2, "The Harbor at Midnight" 36.6 s · −14.2 LUFS

In a midnight storm, Red Tribe claims the 2:1 wheat harbor and lights the beacon. The sea starts glowing and Gemini's robots surface ("THE AIs ARE SURFACING"). They turn the harbor into their trade route ("Two wheat. One ore. Every turn."), and it closes on "Thank you for the lighthouse." with the scoreboard and the series end card. A new night-storm look for the series, with real motion on the opening shots.

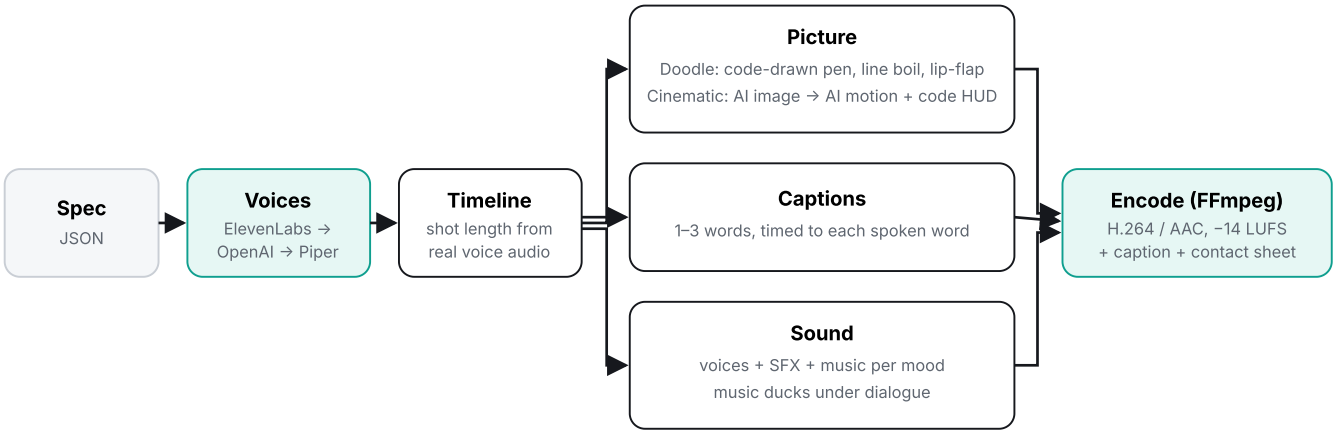
How the existing page styles were reverse-engineered: I downloaded three reference Reels, measured shot cuts and durations, transcribed the dialogue, and studied the fonts, colours and overlay positions frame by frame. I recreated all three as specs (`specs_story/ref_*.json`). Those recreations now serve as the **few-shot examples** that teach Claude the format when it writes new episodes and skits.

3 Architecture

3.1 End-to-end pipeline



3.2 Render engine (inside "Render")



One clock drives picture, captions, mouth movement and sound, so nothing drifts out of sync.

3.3 Provider chain (every external service has a fallback)

Need	First choice → fallback → last resort
Script writing	Claude → retry with validator errors → skip the idea (the batch continues)
Voice	ElevenLabs → OpenAI TTS → local Piper voices (free, offline)
Cinematic image	fal.ai Flux (paid) → Cloudflare (free) → Pollinations → labelled placeholder (status **draft**, never published)
Cinematic motion	fal.ai Kling (paid) → Wan 2.2 on Hugging Face (free quota) → still image with a camera move
Sound effects / music	ElevenLabs SFX / licensed tracks in `assets/music/` → built-in procedural library

Every voice clip, image and video clip is cached by a hash of its inputs, so a re-render (for example after fixing one caption) costs nothing and gives the same result.

4 Workflow and code

4.1 One video, step by step

#	Step	What happens
1	Pick a topic	The topic matrix picks an unused combination, e.g. *poker × GPT × "bluffs with pure air" × escalation gag*. Cinematic episodes also read the series file (story so far, scores) so the season stays consistent.
2	Write the spec	Claude writes a JSON storyboard of shots, with each shot's visual, spoken lines (with tone), sound effects and overlays, by calling a `submit_spec` tool. Few-shot examples are the recreations of the page's reels.
3	Validate and judge	Gate 1 (rules) and Gate 2 (judge). Errors and judge notes go back to Claude; accepted specs are saved to disk.
4	Voice	Each line is voiced with that character's voice; word timings come back with the audio.
5	Timeline	Shot lengths come from the real voice durations; captions are chunked and timed per word.
6	Picture	Doodle: drawn frame by frame by code. Cinematic: AI image and motion per shot, with code-drawn text and HUD on top.
7	Sound and encode	Voice, SFX and ducked music are mixed; FFmpeg encodes to Instagram's format with two-pass loudness and a peak limiter.
8	QC and manifest	Gate 4 checks, a contact sheet, and a row in `manifest.csv` with the status and reason.

4.2 What a spec looks like (shortened)

```
{ "id": "doodle_001_nothing_bluff", "format": "doodle", "game": "poker",
  "hook": {"title": "POV:", "text": "The AI that bluffs with pure air"},
  "cast": {"wren": {"kind": "human", "voice": "human_male"},
           "chip": {"kind": "ai", "brand": "GPT", "voice": "robot_calm"}},
  "shots": [
    {"visual": {"type": "table", "board": ["9♠","4♦","J♣","3♥","8♦"]},
     "lines": [{"who": "wren", "say": "I folded kings. Okay. What did you have?"}]},
    {"visual": {"type": "closeup", "who": "chip", "expression": "neutral"},
     "lines": [{"who": "chip", "say": "Nothing."}]},
    {"visual": {"type": "card_reveal", "board": [...], "hidden": ["7♣","2♦"]},
     "music": 0.0, "sfx": [{"at": 0.4, "cue": "card_flip"}], "dur": 2.6}, ... ],
  "caption": "He folded pocket kings... Play poker vs frontier AI at gghuman.ai" }
```

Visuals, voices, sound effects and overlays come from a **fixed menu** (e.g. 10 doodle shot types, 9 HUD overlays, 24 sound cues), so the renderer never has to interpret free text. Only cinematic image descriptions go to an image model.

4.3 Project structure

```
run.py                  CLI: render · batch · ideate · story-ideate · story-factory
pipeline/
  spec.py · render.py · scenes.py · audio.py · ideate.py   "Said vs Thought" format
  qc.py                 post-render checks + contact sheets (all formats)
  story/
    spec.py             storyboard contract: vocabulary, limits, game-legality rules
    voice.py            TTS provider chain, cache, word timings, robot voice effect
    timeline.py         shot lengths from voice audio, caption chunking
    doodle.py           hand-drawn renderer (pen, characters, props, camera)
    genai.py            image / motion providers with fallbacks and cache
    cinematic.py        footage, transitions, faction captions, HUD, end card
    sound.py            SFX library, music beds, ducking, mix
    ideate.py           Claude writer + judge + one revision round
series/ · specs*/ · generated_video/   series bible · scripts (ref_* = few-shot examples) · final MP4s
```

5 Checking output quality

Four gates, **cheapest first**, so a weak idea is rejected before anything is paid for or rendered.

Gate	When	What it checks
1. Rule validator	Before any spend	Required fields; only allowed shot types, voices and effects; text length limits (they are the screen's safe zones); **game legality** (real card codes with no duplicates, dice 1–6, the robber never on a 7, valid factions); banned phrases (no "free", no gambling-for-money wording, no insults); estimated length in range; caption mentions gghuman.ai; not too similar to an existing video.
2. LLM judge	Before render	A separate Claude call scores *hook, comedy/drama, game accuracy, clarity, brand fit* from 1 to 5. Every score must be ≥ 4. A rejected draft gets **one revision** with the judge's notes; if it still fails, it is saved to `_rejected/` for review.
3. Layout check	During render	Any caption, title or sign that doesn't fit its box stops that render with an exact message ("caption 'X' is 1,020 px wide"). Text is never silently shrunk; the spec goes back to the writer.
4. Automated QC	After encode	Format (1080×1920, H.264/yuv420p, audio present); length per format; loudness −16.5…−11.5 LUFS and true peak; black frames (end card excluded); frozen picture; first caption within 1.5 s (hook pacing); placeholder footage → status **draft**, never published.

Human in the loop

Each video gets a **contact sheet** (one frame per shot) for a ~5-second review. Review every video in the first batch, then ~10% at random. Nothing is published without status `ok` and approval.

Closing the loop with real data

After launch, feed Instagram metrics (3-second hold, completion, profile taps, link clicks) back per topic and joke structure, and reweight the topic matrix toward what works.

Results on the delivered videos

Video	Length	Loudness	QC	Judge (min score)
Poker vs GPT	17.5 s	−13.2 LUFS	pass	n/a (hand-written reference)
Last Vote vs Gemini	17.6 s	−13.1 LUFS	pass	n/a (hand-written reference)
Doodle: "Nothing." bluff	22.5 s	−13.7 LUFS	pass	4 / 5 (game accuracy 5)
Doodle: polite robber	16.3 s	−14.1 LUFS	pass	n/a (hand-written)
Cinematic EP 2	36.6 s	−14.2 LUFS	pass	n/a (hand-written)

6 Handling failures

Principle: **one failure never stops the batch, and every fallback is visible** in the manifest.

Failure	Handling
Claude returns an invalid or over-long spec	Exact validator errors are sent back as a tool error; up to 3 attempts, then the idea is skipped.
Judge rejects the script	One revision using the judge's notes; then dropped and archived in `_rejected/`.
Script too similar to an earlier one	Treated as a validation error, so Claude writes a different angle.
Text doesn't fit the layout	Render stops for that video; the spec goes back to Claude once with the exact error.
Voice / image / video service down, out of quota or unauthorised	Next provider in the chain (section 3.3). The cache keeps all finished work, so re-runs only fill gaps.
No real footage available	Placeholder frames, status **draft**, never published.
Render or FFmpeg crash	Caught per video and logged in `manifest.csv` with the error; other videos continue.
Crash mid-batch / API outage	Specs are written to disk as they're created; `batch` is resumable and skips everything already `ok`.
QC fails (loudness, black frames, length)	Status `qc_failed` with the reason; goes to the review queue, not to Instagram.

Real incidents during this project, caught by these layers

What happened	What the system did / what I changed
The Claude API stopped accepting "you must call this tool"	Writer switched to automatic tool choice plus an explicit instruction and one nudge; the validator still guarantees the structure.
The judge rejected 8 drafts in a row, most missing by one point	Added the revision round with judge notes; the next run passed on the first attempt.
Some built-in ElevenLabs voices had been retired	Remapped each voice role to the voices available on the account.
The image token lacked the required permission; the free fallback started charging	Clear error message and fallback; renders became drafts instead of failing silently.
The free video quota ran out after 2 clips	Remaining shots fell back to stills with camera moves; finished clips stay cached for the next run.
A sandboxed FFmpeg refused stop signals	A per-video crash was logged in the manifest, then fixed to let the process exit on its own.

7 Automating 100 videos

7.1 How

```
python run.py story-factory --format doodle    --n 70 --workers 4
python run.py story-factory --format cinematic --n 30 --series settler_rush
python run.py batch --specs specs --out out --workers 4     # Said vs Thought
```

- **Each job is a pure function: spec.json → MP4.** It runs in parallel on one machine or fans out on any job queue or cloud workers.
- **Variety without repeats:** doodle has 16 AI behaviours × 4 joke structures × 5 AI brands (320 combinations); Said vs Thought has 3 games × 9 AI opponents × 4 deceptions × 5 hook formulas (540). A similarity check rejects near-duplicate ideas.
- **Series continuity:** cinematic episodes are written *in order* (each one reads and updates the season's story and scoreboard), then rendered in parallel.
- **Resumable and idempotent:** rerunning a batch only does what's missing; the cache prevents paying twice.

7.2 Time and cost (measured on an 8-core laptop, no GPU)

Stage	Per video	100 videos (e.g. 70 doodle / 30 cinematic)
Writing + judge (+ revision)	3–6 Claude calls, ~1–3 min	About 1 hour with parallel writers; budget 3–4 ideas per accepted video
Voices (ElevenLabs)	150–450 characters	25–35k characters: above the free 10k/month, so a paid plan
Doodle / Said vs Thought render	~60–90 s	~30–45 min with 3–4 parallel workers
Cinematic images	~11 images × ~10 s	~330 images; fits free daily allowances when spread over days
Cinematic motion	~50 s per clip (free GPU)	**Bottleneck:** ~300 clips vs a few free clips per day, so a paid video model (roughly a few dollars per episode) or motion on hero shots only
Cinematic composite + encode	~1.5–2.5 min	~1 hour

7.3 Production plan

#	Step
1	**Write all 100 scripts first** (overnight). They're cheap and are checked before any render spend.
2	**Render the code-drawn formats immediately** (doodle, Said vs Thought); the only external cost is voices.
3	**Cinematic:** paid video model for full motion, or free motion on 3–4 hero shots plus a nightly job that fills in more clips as quota resets.
4	**Review gate:** contact-sheet review (all of the first batch, then ~10%); only `ok` plus approved videos move on.
5	**Publish** 1–3 per day via the Instagram Graph API (container from a hosted MP4 + caption → poll → publish), scheduled from the manifest.
6	**Learn:** weekly metrics per topic and joke structure → reweight the topic matrix.

8 What I'd do next with access to gghuman.ai

- **Use real game logs.** Feed the AIs' actual chat lines and reasoning traces into the specs, so every video is a true story ("this actually happened on GGhuman").
- **Full motion for every cinematic shot** with a paid image-to-video model, and consistent recurring characters across episodes (reference images per faction).
- **A/B hooks:** render 2 hooks per spec and let the first 24 h of reach choose the winner.
- **Publishing automation** through the Instagram Graph API, plus a small dashboard over the manifest for the review queue.
- **Licensed music library** per mood (already supported by dropping files into `assets/music/`).

Lessons on getting AI to do creative work reliably

1. **Give the model a small, strict output contract** (a JSON spec with a fixed vocabulary), not open-ended pixels or free text.
2. **Validate everything mechanical with code**, and send the model the *exact* errors; models fix precise feedback reliably.
3. **Judge taste with a rubric**, and give the writer one chance to revise with the judge's notes instead of throwing work away.
4. **Keep the deterministic parts deterministic:** text, layout, timing and brand elements belong in code.
5. **Assume every external service will fail:** provider chains, caching, resumable batches and visible "draft" states turn outages into delays instead of bad posts.

Source: `gg-reels/` · run `python run.py --help` · final videos and captions in `generated_video/`